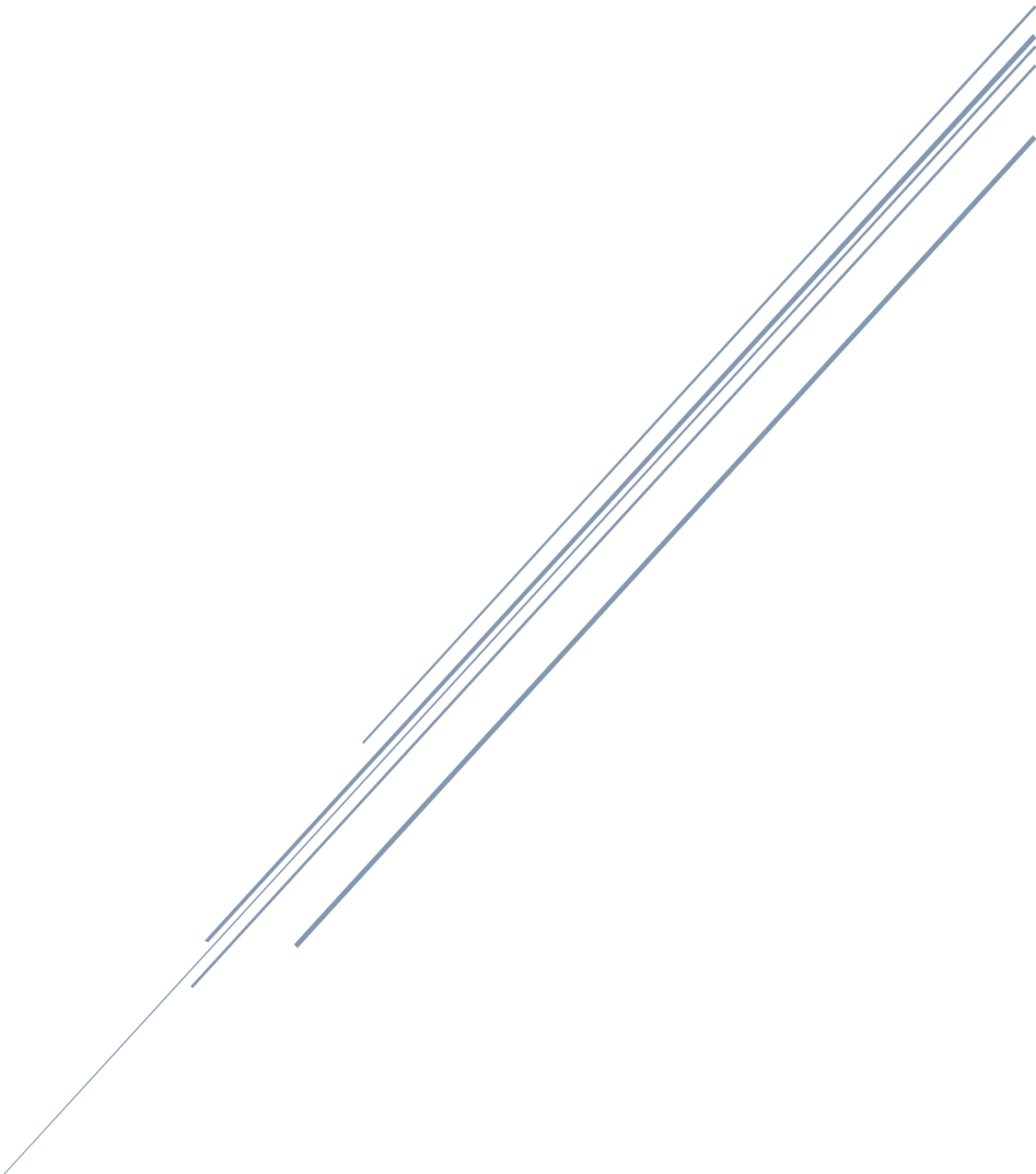


SECURE TICKETING & RESERVATION API

SYSTEM DESIGN DOCUMENT



Version History Page	2
1. Introduction	3
2. Project Plan	3
2.1 Clarify Requirements	3
2.1.1 Functional Requirements	3
2.1.2 Non-Functional Requirements	4
2.2 Capacity & Load Estimation	4
2.3 API Design	6
2.4 Database Design	6
2.4.1 Data Model	7
2.5 High-Level Architecture Design	7
2.5.1 Architectural Decisions	8
2.5.2 Diagrams	9
2.6 Dive Into Key Components	11
2.7 Scalability, Fault Tolerance & Reliability	12
1. Scalability	12
2. Fault Tolerance	13
3. Reliability	13
4. Observability	13
3. Developer Guide	14
3.1 Project Structure	14
3.2 Requirements	15
3.3 How to Run the Application	15
3.3.1 Start Infrastructure	15
3.3.2 Build and Run	16
3.3.3 Open Swagger UI	16
3.3.4 Seed Users	17
3.3.5 Authentication Flow	17
3.3.6 Example API Calls	18
3.3.7 Actuator	18
3.4 Public API Documentation	19
3.5 Testing	20
3.5.1 Test Coverage	21
3.6 Static & Security Code Analysis	22
3.6.1 Static Code Analysis (PMD)	22
3.6.2 Secure Code Analysis (SpotBugs + Find Security Bugs)	23
4. Future Improvements	24

Version History Page

Version	Author	Reason	Date
1.0	Mert Karaçam	Initial Draft	14.03.2026

1. Introduction

The Secure Ticketing & Reservation API is a backend service built to handle event management and ticket reservation under real-world constraints. The system addresses three core problems that any production ticketing platform must solve: concurrent users booking the last available seat at the same moment, duplicate requests arriving due to network retries, and the need to enforce strict access boundaries across different user roles.

The service exposes a RESTful API secured with stateless JWT authentication, role-based authorization enforced at the method level, and a concurrency model that uses pessimistic database-level locking rather than application-level synchronization. Every reservation write passes through an idempotency layer that detects duplicate submissions and replays the original response without re-executing the business logic.

This document serves as a complete reference for developers working with this codebase. It covers the reasoning behind key architectural decisions, the data model, the API surface, security mechanisms, testing strategy, and setup instructions.

2. Project Plan

2.1 Clarify Requirements

Before designing or implementing The Secure Ticketing & Reservation API, the first step was to clearly understand what the system is expected to do and under which constraints it must operate. The requirements were grouped into two categories: **functional** and **non-functional**.

2.1.1 Functional Requirements

1. User Management

- Users can register with an email and password.
- Three roles are supported: ADMIN, ORGANIZER and CUSTOMER.
- Login returns a short-lived access token (15 minutes) and a long-lived refresh token (7 days).
- Access tokens can be renewed using the refresh endpoint without re-authentication.

2. Event Management

- Organizers can create, update and publish events.
- Events have a title, venue, start/end time and a total capacity.
- Published events are visible to all users without authentication.
- Published events cannot be modified.

3. Reservation Engine

- Customers can create, confirm and cancel reservations.
- The system must prevent overselling under concurrent booking attempts.
- Reservations in CONFIRMED status cannot be cancelled.
- All reservation actions are recorded in an audit log.

4. Idempotency

- POST requests that include an Idempotency-Key header are processed exactly once.
- Duplicate requests with the same key return the original response without re-processing.

5. Rate Limiting

- The login endpoint is rate-limited to 10 requests per minute.
- Clients exceeding the limit receive a 429 Too Many Requests response.

6. Audit Logging

- Every reservation lifecycle action is written to an audit log asynchronously.
- Audit records are preserved even if the triggering transaction rolls back.

2.1.2 Non-Functional Requirements

1. Security

- Passwords are stored as BCrypt hashes with cost factor 12.
- JWT tokens are signed with HMAC-SHA256 using a configurable secret.
- Hash comparison for idempotency keys uses constant-time comparison to prevent timing attacks.
- CORS is configured to allow cross-origin requests from frontend applications.

2. Concurrency Safety

- The booking path uses pessimistic locking (SELECT FOR UPDATE) at the database level.
- This ensures correctness in multi-instance deployments, unlike JVM-level synchronization.

3. Observability

- Spring Boot Actuator exposes health and info endpoints.
- Database and Redis health details are visible to authenticated users.
- Audit logs provide a complete history of reservation activity.

4. Testability

- All service and controller layers are independently unit-testable.
- Concurrency correctness is verified with a dedicated multi-threaded integration test.
- Test coverage exceeds 90% across the codebase.

5. Scalability

- Database-level locking supports horizontal scaling without shared in-process state.
- Redis is configured for distributed rate limiting in future multi-instance deployments.
- Cache layer (Redis) reduces repeated database reads for public event listings.

2.2 Capacity & Load Estimation

The Secure Ticketing & Reservation API is designed to handle event-driven traffic patterns where load spikes sharply around popular event releases and drops off between them. The following estimates are approximations to justify certain design choices such as pessimistic locking, Redis caching and idempotency key storage.

1. Expected Traffic Profile

- Normal load: ~50 concurrent users browsing and reserving tickets
- Peak load: ~500 concurrent users during high-demand event releases
- Reservation burst: up to 200 booking requests per second at peak
- Read/write ratio: approximately 80% reads (event browsing) to 20% writes (reservations, logins)

2. Request Volume Estimates

Endpoint	Estimated RPS (peak)	Notes
GET /api/events/public	150	Cached in Redis, 5-min TTL
POST /api/reservations	200	Pessimistic lock, DB write
POST /api/auth/login	10 (limited)	Rate limited by Resilience4j
PUT /api/reservations/{id}/confirm	80	DB write + audit log
GET /api/reservations/my	50	Authenticated, DB read

3. Database Load

- Each reservation write generates two DB operations: a locked event row read and a reservation insert.
- Audit log writes are async and do not impact the main transaction latency.
- Idempotency key storage adds one read and one write per unique POST request, with negligible overhead.
- With 200 reservation RPS at peak, the system produces approximately 400 DB operations per second on the write path.

4. Storage Estimation

- Average reservation record size: ~400 bytes
- Average audit log record size: ~300 bytes
- Idempotency key record (with response body): ~2-5 KB, cleaned up after TTL expiry
- At 1 million reservations: approximately 400 MB for reservations + 300 MB for audit logs
- This is well within the range of a standard PostgreSQL deployment without partitioning

5. Caching Impact

- Public event listings are cached in Redis with a 5-minute TTL.
- At 150 read RPS for event browsing, caching eliminates approximately 90% of repeated DB queries for the same result set.
- Cache invalidation is triggered on every event create, update, or publish operation via @CacheEvict.

6. Concurrency Safety Under Load

- Pessimistic locking serializes writes to the same event row. Under extreme concurrency (200 booking RPS for a single event), this creates a queue at the DB level.
- PostgreSQL can handle this queue efficiently for moderate event capacities. For very high-demand events, a ticket reservation queue (Kafka + outbox pattern) would be the recommended next step.
- The OversellConcurrencyTest validates correctness under 20 concurrent threads, confirming the locking strategy works as intended.

2.3 API Design

The API follows RESTful conventions with predictable URL patterns, HTTP verb semantics, and consistent error envelopes. All endpoints are documented via SpringDoc OpenAPI and accessible through Swagger UI.

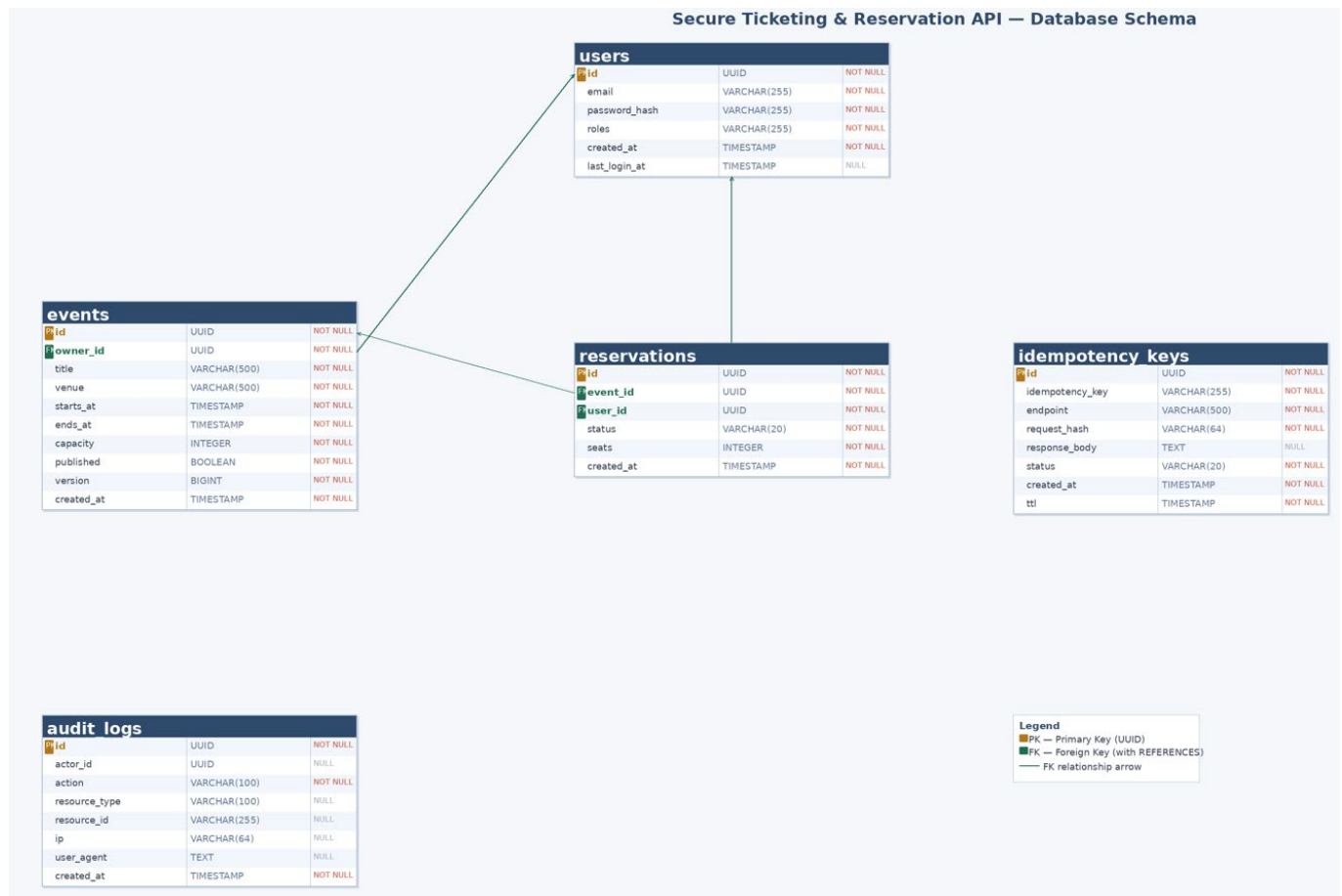
Method	Path	Role	Description
POST	/api/auth/register	Public	Register a new user
POST	/api/auth/login	Public	Authenticate, receive JWT tokens
POST	/api/auth/refresh	Public	Refresh access token
GET	/api/events/public	Public	Browse published events (cached)
GET	/api/events/{id}	Authenticated	Get event by ID
GET	/api/events	ORGANIZER / ADMIN	List events (paginated)
POST	/api/events	ORGANIZER / ADMIN	Create a draft event
PUT	/api/events/{id}	ORGANIZER / ADMIN	Update a draft event
POST	/api/events/{id}/publish	ORGANIZER / ADMIN	Publish a draft event
POST	/api/events/{eventId}/reservations	CUSTOMER / ADMIN	Create a reservation
POST	/api/reservations/{id}/confirm	Authenticated	Confirm a pending reservation
POST	/api/reservations/{id}/cancel	Authenticated	Cancel a reservation

2.4 Database Design

The data model uses five tables. Each table maps to a single bounded concept in the domain. No denormalization is used, and foreign key constraints are applied where referential integrity is required.

Table	Purpose
users	Stores credentials, roles, and last login timestamp
events	Event definitions with capacity, dates, and published flag
reservations	Tracks reservation state (PENDING, CONFIRMED, CANCELLED)
audit_logs	Append-only log of every reservation action
idempotency_keys	Stores request hashes and response bodies for deduplication

2.4.1 Data Model



2.5 High-Level Architecture Design

The application follows a layered architecture: filter chain, controller, service, and repository. Cross-cutting concerns such as security, audit logging, and idempotency are handled outside the business logic layer using Spring Security filters and AOP aspects.

Request lifecycle:

- IdempotencyFilter intercepts POST requests and checks for a matching key. If found and hash matches, the stored response is replayed immediately.
- JwtAuthenticationFilter validates the Bearer token and populates the SecurityContext.
- Resilience4j RateLimiter enforces the login request budget.
- Controllers delegate to services. Role enforcement happens at the service method boundary via `@PreAuthorize`.
- ReservationService acquires a pessimistic lock on the event row before modifying seat counts.
- AuditAspect fires asynchronously after each reservation write, using a separate transaction to ensure the audit record is committed independently.

Infrastructure layer:

- PostgreSQL is the primary database for all persistent state.
- Redis is used as a cache for public event listings (TTL: 5 minutes) and is pre-configured for distributed rate limiting.
- Flyway manages all schema migrations, ensuring the database state is always version-controlled.
- H2 in-memory database is used exclusively for the test suite, allowing tests to run without external dependencies.

ADR-01: Pessimistic Locking for Oversell Prevention

Pessimistic locking (SELECT FOR UPDATE) is applied to the event row during reservation creation. This provides a deterministic guarantee that only one transaction can modify the available capacity at a time. Optimistic locking with retry was considered but rejected due to unpredictable latency under contention and the complexity of retry orchestration. Because the event row is the natural contention point in this domain, a database-level lock is both simpler and more portable across deployment topologies.

ADR-02: Idempotency via Database

Idempotency keys and response bodies are stored in PostgreSQL rather than Redis. The database is already the system of record, and storing idempotency data there provides ACID guarantees on the key-response pair at no additional operational cost. Redis was deferred to the scale-out phase where TTL-based expiry and lower latency would become worthwhile.

ADR-03: Async AOP Audit Logging with REQUIRES_NEW

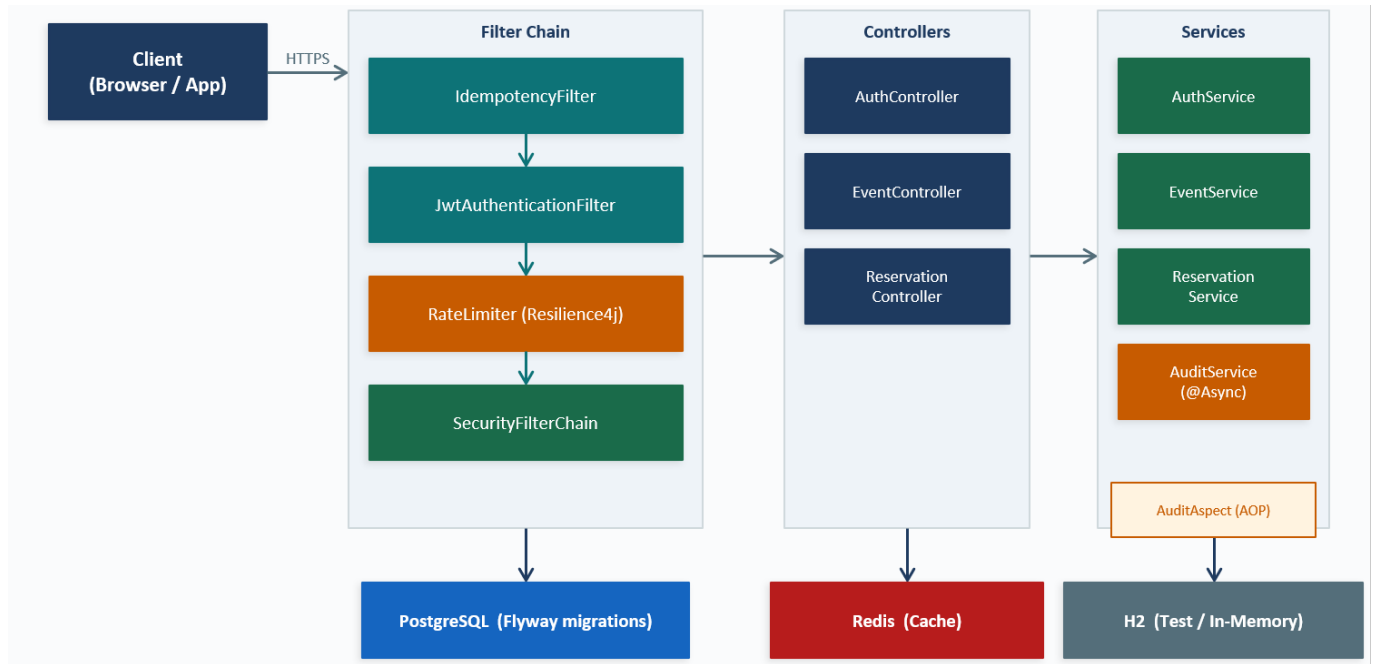
Audit records are written through an AOP aspect using `@Async` and `REQUIRES_NEW` transaction propagation. This ensures that audit writes do not block the main business transaction and are committed independently, even if the parent transaction rolls back. The result is a complete audit trail that captures both successful and failed operations.

ADR-04: Native SQL for Published Event Queries

The `findPublished` repository method uses a native SQL query rather than JPQL. PostgreSQL requires explicit type casts on nullable parameters in prepared statements, which JPQL's Hibernate dialect does not handle transparently. Native SQL with `CAST` expressions produces identical behavior across both the H2 test database and the PostgreSQL production database without requiring dialect-specific configuration.

2.5.2 Diagrams

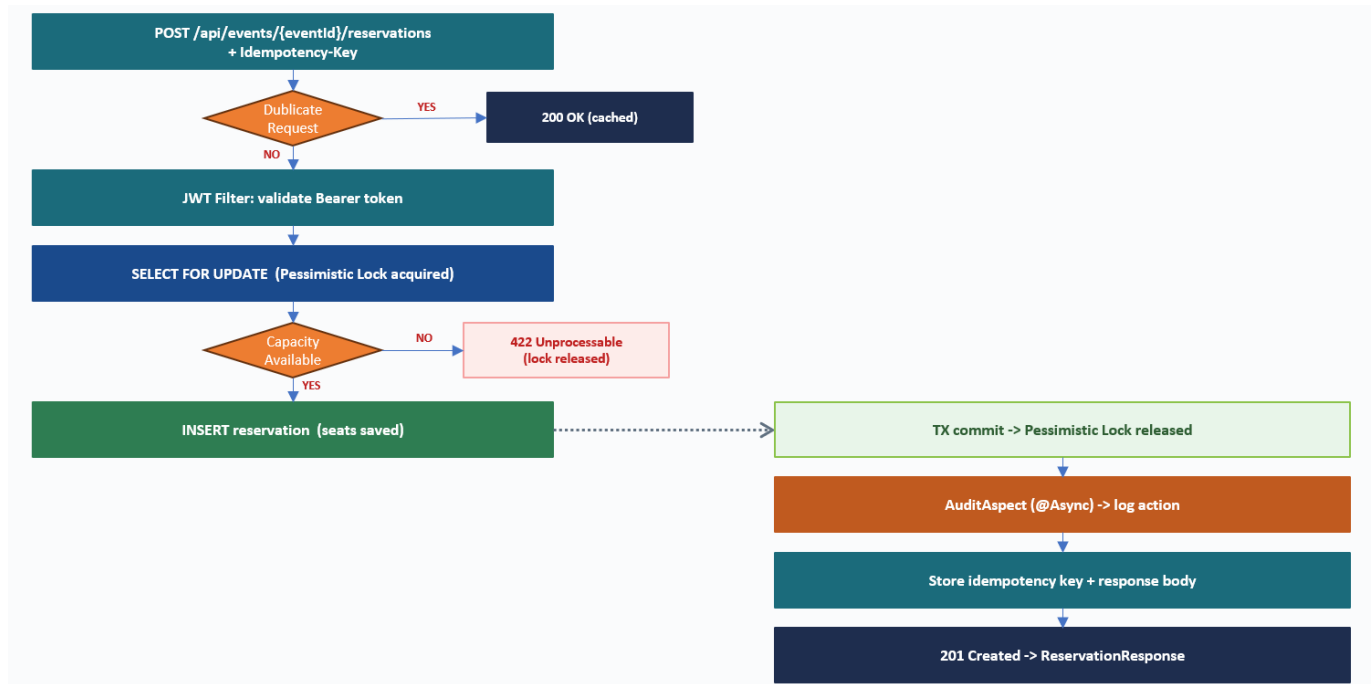
2.5.2.1 High-Level Architecture



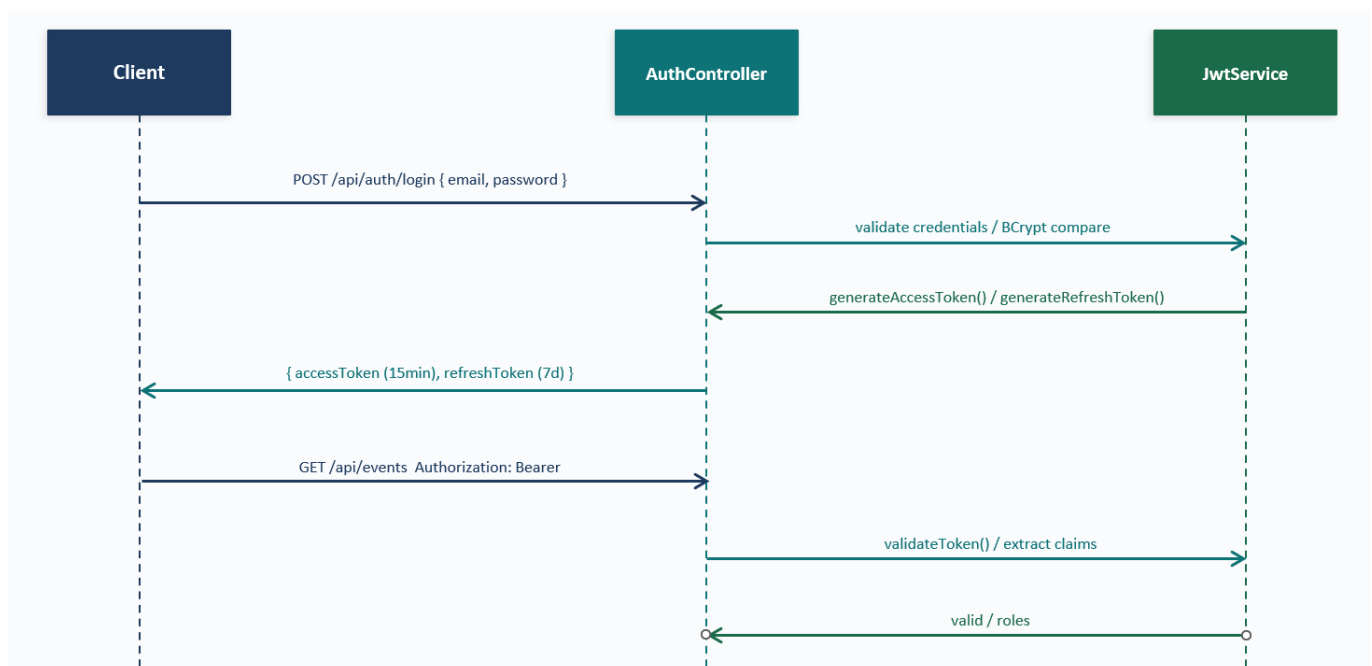
2.5.2.2 Request Lifecycle-Filter Chain



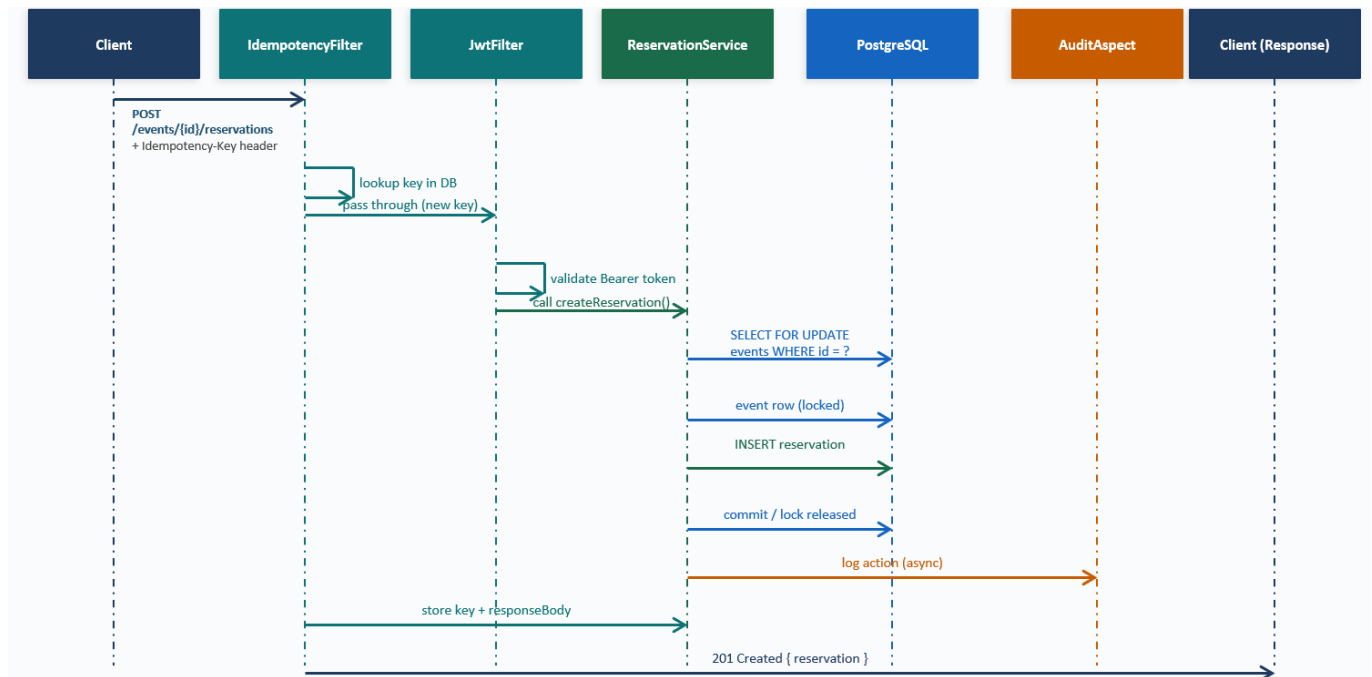
2.5.2.3 Reservation Flows-Pessimistic Logging



2.5.2.4 JWT Authentication Flow



2.5.2.5 Sequence Diagram- Create Reservation



2.6 Dive Into Key Components

This section examines each major component of the Secure Ticketing and Reservation API in detail, explaining its responsibilities, design decisions, and how it interacts with the rest of the system.

JwtAuthenticationFilter

Runs on every request before the controller. Reads the Bearer token from the Authorization header, validates it via `JwtService`, and loads the user into the `SecurityContext`. If the token is missing or invalid it just passes through the filter chain handles the 401 downstream. Open endpoints like `/api/auth/**` are unaffected.

IdempotencyFilter

Applies only to POST requests with an Idempotency-Key header. It hashes the request body and checks the `idempotency_keys` table. If a completed record exists with the same hash, the cached response is returned immediately without touching the controller. New requests get a `PROCESSING` record first; after the response is written, the record is updated to `COMPLETED` with the response body stored.

SecurityConfig

Sets up the Spring Security filter chain: stateless sessions, CSRF disabled, `JwtAuthenticationFilter` added before the default auth filter. Route rules are explicit `/api/auth/**` and `/api/events/public` are public, everything else requires authentication. Fine-grained role checks are handled per method with `@PreAuthorize`.

JwtService

Generates access tokens (15 min) and refresh tokens (7 days) signed with HS512. Both carry `userId`, email, roles and a type claim. The type claim is how the system tells them apart a refresh token cannot be used as a bearer token for protected endpoints.

AuthService

Handles register, login, and refresh. Register hashes the password with BCrypt (cost 12) and stores the user. Login delegates to Spring's AuthenticationManager, records last login, and returns a token pair. Refresh validates the incoming refresh token, confirms it is the right type, and issues new tokens.

EventService

Manages the event lifecycle. Create sets the owner to the authenticated user and marks the event as draft. Update blocks edits on already-published events. Publish checks the event is still in the future. Listing is role-aware: admins see all, organizers see their own. The public discovery endpoint runs a native SQL query with optional date and text filters.

ReservationService

The most concurrency-sensitive part. It acquires a PESSIMISTIC_WRITE lock on the event row (SELECT FOR UPDATE), sums active seats, and compares against capacity — all inside a single transaction. If capacity is exceeded, a BusinessException is thrown and the lock is released on rollback. Confirm and cancel check that the caller is the reservation owner, the event organizer, or an admin.

AuditAspect

An @AfterReturning AOP pointcut on all state-changing service methods. It captures actor, action, resource ID, IP, and User-Agent, then calls AuditService.log() which runs @Async in a REQUIRES_NEW transaction. Audit failures are silently caught so they never roll back the main operation.

GlobalExceptionHandler

A @RestControllerAdvice that maps exceptions to structured error responses with status, type, message, and timestamp. Covers 400 validation errors, 401 bad credentials, 403 forbidden, 404 not found, 409 conflicts, 422 business rule violations, 429 rate limit, and 500 for anything unexpected. Stack traces never reach the client.

Resilience4j Rate Limiter

Applied to the login endpoint via @RateLimiter annotation. Configured at 10 requests per minute with zero wait time. Rejected requests hit a fallback that throws RateLimitException, which the exception handler maps to 429. Keeping this as an annotation makes it easy to extend with circuit breaking or retry in a future microservice setup.

2.7 Scalability, Fault Tolerance & Reliability

1. Scalability

The service is stateless by design. No session data is kept in memory, so running multiple instances behind a load balancer works without any extra coordination. The main scalability points are:

- **Horizontal scaling:** add instances freely. JWT validation, idempotency checks, and cache all live outside the process.
- **Pessimistic locking:** the SELECT FOR UPDATE on the events row serializes concurrent reservation writes at the database level, regardless of how many instances are running.
- **Redis cache:** the public events listing is cached so repeated reads don't hit the database. Cache is invalidated on any write.
- **Rate limiting:** currently Resilience4j in-process. Replacing it with a Redis-backed limiter would extend protection across multiple instances with no controller changes.

2. Fault Tolerance

Each component is designed to fail without bringing anything else down with it.

Scenario	What happens
Invalid JWT	The filter catches the exception and passes through. Spring Security returns 401 no crash, nothing exposed.
Duplicate request	Idempotency filter returns the cached response before the controller runs.
Concurrent oversell	Only one transaction gets past the capacity check. The rest receive 422. No partial data is written.
Audit failure	AuditAspect swallows errors silently. The main transaction is not affected.
Redis down	Cache misses fall through to PostgreSQL. The application keeps working, just at higher DB load.

3. Reliability

- **Transactions:** every write is wrapped in `@Transactional`. If anything fails mid-operation, the whole thing rolls back.
- **Optimistic locking on events:** the `@Version` field on Event prevents silent last-write-wins overwrites under concurrent edits.
- **Schema consistency:** Flyway applies migration scripts in order with checksum tracking, so the schema always matches the deployed version.
- **Audit trail:** AuditLog rows are insert-only. They are never updated or deleted.
- **Error responses:** GlobalExceptionHandler maps every exception to a clean JSON response. Stack traces stay on the server.

4. Observability

The application exposes enough information to diagnose problems without leaking internals.

Component	Level	What is logged
JwtAuthenticationFilter	DEBUG	Invalid token events no token content in logs
GlobalExceptionHandler	ERROR	Unhandled exceptions with full stack trace
AuditAspect	Async DB	Actor, action, resource ID, IP and User-Agent for every state change
Spring Actuator	HTTP	/actuator/health DB and Redis details shown to authenticated users only
Swagger / OpenAPI	HTTP	Full API docs at /swagger-ui.html with JWT bearer auth

The metrics endpoint is excluded from public exposure on purpose. Health details are only visible to authenticated callers so internal system state stays private.

3. Developer Guide

3.1 Project Structure

com.ticketing.reservation

controller/

- AuthController - register, login, refresh token endpoints
- EventController - create, update, publish, list events
- ReservationController - create, confirm, cancel reservations

service/

- AuthService - registration, login, refresh token logic
- EventService - event lifecycle, ownership checks, public discovery
- ReservationService - pessimistic locking, seat validation, status transitions
- AuditService - async audit log writes

security/

- JwtService - token generation and validation (HS512)
- JwtProperties - token secrets and expiry loaded from application.yml
- UserPrincipal - Spring Security UserDetails wrapper
- UserDetailsServiceImpl - loads user by email for Spring auth

filter/

- JwtAuthenticationFilter - validates Bearer token on every request
- IdempotencyFilter - deduplication for POST requests with Idempotency-Key header

aspect/

- AuditAspect - @AfterReturning pointcut that logs all state-changing operations

config/

- SecurityConfig - filter chain, route authorization rules, CORS, BCrypt bean
- OpenApiConfig - Swagger/OpenAPI setup with JWT bearer auth support
- RedisConfig - Redis template configuration

domain/entity/

- User, Event, Reservation, AuditLog, IdempotencyKey - JPA entities

domain/enums/

- ReservationStatus (PENDING, CONFIRMED, CANCELLED), Role (ADMIN, ORGANIZER, CUSTOMER)

dto/

- request/ - RegisterRequest, LoginRequest, RefreshRequest, EventRequest, ReservationRequest
- response/ - Responses (nested records: AuthResponse, EventResponse, ReservationResponse, ErrorResponse, PageResponse)

repository/

- UserRepository, EventRepository, ReservationRepository, AuditLogRepository, IdempotencyKeyRepository

exception/

- GlobalExceptionHandler - maps all exceptions to structured ErrorResponse
- BusinessException, ConflictException, ResourceNotFoundException, RateLimitException

db/migration/

- V1__init_schema.sql - all tables and indexes
- V2__seed_users.sql - admin, organizer, customer seed users

3.2 Requirements

- **Java** 21+
- **Spring Boot** 4
- **Maven** 3.9+
- **Docker Desktop** (for PostgreSQL and Redis via docker-compose)
- **Internet** — required on first build to download dependencies
- **IDE**: Eclipse, IntelliJ IDEA, or VS Code

3.3 How to Run the Application

3.3.1 Start Infrastructure

The application requires PostgreSQL and Redis. Both are defined in docker-compose.yml at the project root.

Open a terminal in the project root and run:

```
docker-compose up -d
```

This starts:

- **ticketing-postgres** — PostgreSQL 16 on port 5432
- **ticketing-redis** — Redis 7 on port 6379

On first startup, Flyway runs V1__init_schema.sql and V2__seed_users.sql automatically. No manual DB setup is needed.

3.3.2 Build and Run

Option A — Terminal

```
mvn clean install
mvn spring-boot:run
```

The application starts on <http://localhost:8080>

Option B — IDE

- **Open the project in your IDE**
- **Right-click pom.xml** → Run As → Maven Install
- **Open SecureTicketingReservationApiApplication.java** → Run As → Java Application

3.3.3 Open Swagger UI

Once the application is running, open:

```
http://localhost:8080/swagger-ui/index.html
```

The screenshot displays the Swagger UI for the 'Secure Ticketing & Reservation API' (version 1.0.0, OAS 3.1). The interface includes a top navigation bar with the Swagger logo, a search bar containing '/api-docs', and an 'Explore' button. Below the header, the API title is followed by a description: 'Event ticketing platform with JWT authentication, role-based authorization, idempotent reservations and oversell prevention.' It also lists roles (ADMIN, ORGANIZER, CUSTOMER) and the authentication flow (Register → Login → Bearer token). A 'Servers' dropdown shows 'http://localhost:8080 - Generated server url', and an 'Authorize' button is present.

The API endpoints are organized into three sections:

- Authentication & Authorization** (Register, login and token refresh):
 - POST `/api/auth/register`: Register a new user
 - POST `/api/auth/refresh`: Refresh access token
 - POST `/api/auth/login`: Authenticate and receive JWT tokens
- Event Management** (Event management endpoints):
 - GET `/api/events/{id}`: Get event by ID
 - PUT `/api/events/{id}`: Update an event
 - GET `/api/events`: List events
 - POST `/api/events`: Create a draft event
 - POST `/api/events/{id}/publish`: Publish a draft event
 - GET `/api/events/public`: Browse published events - no auth required
- Reservations & Discovery** (Reservation lifecycle endpoints):
 - POST `/api/reservations/{id}/confirm`: Confirm a PENDING reservation
 - POST `/api/reservations/{id}/cancel`: Cancel a reservation
 - POST `/api/events/{eventId}/reservations`: Create a reservation - requires Idempotency-Key header

From Swagger UI you can:

- **Register a user** and log in to get a JWT token
- **Click Authorize** and paste the access token to authenticate all requests
- **Create and publish events** (ORGANIZER or ADMIN role required)
- **Make reservations** (CUSTOMER role)
- **Confirm or cancel reservations**

3.3.4 Seed Users

V2__seed_users.sql pre-inserts three users for immediate testing:

Email	Password	Role
admin@ticketing.com	Admin123!	ADMIN
organizer@ticketing.com	Organizer123!	ORGANIZER
customer@ticketing.com	Customer123!	CUSTOMER

3.3.5 Authentication Flow

All protected endpoints require a Bearer token in the Authorization header.

1. Register (optional — seed users already exist)

```
POST /api/auth/register
```

```
{ "email": "test@example.com", "password": "Test@12345!", "role": "CUSTOMER" }
```

2. Login

```
POST /api/auth/login
```

```
{ "email": "organizer@ticketing.com", "password": "Organizer123!" }
```

Response:

```
{ "accessToken": "...", "refreshToken": "...", "tokenType": "Bearer", "expiresIn": 900 }
```

3. Authorize in Swagger

Click the Authorize button, paste the accessToken value, click Authorize. All subsequent requests will include the token automatically.

4. Refresh token

```
POST /api/auth/refresh
```

```
{ "refreshToken": "..." }
```

Returns a new access token without requiring login credentials again.

3.3.6 Example API Calls

Create an event (ORGANIZER)

```
POST /api/events
Header: Idempotency-Key: event-001
{
  "title": "Spring Boot Workshop",
  "venue": "Istanbul Tech Hub",
  "startsAt": "2026-06-15T14:00:00Z",
  "endsAt": "2026-06-15T18:00:00Z",
  "capacity": 100
}
```

Publish the event

```
POST /api/events/{id}/publish
```

Make a reservation (CUSTOMER)

```
POST /api/events/{eventId}/reservations
Header: Idempotency-Key: res-001
{ "seats": 2 }
```

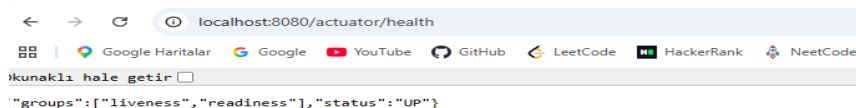
3.3.7 Actuator

Spring Boot Actuator is enabled and exposes two endpoints:

- GET /actuator/health - returns application health status including database and Redis connectivity. Authenticated users receive full component details; unauthenticated requests return a simple UP/DOWN status.
- GET /actuator/info - returns application metadata.

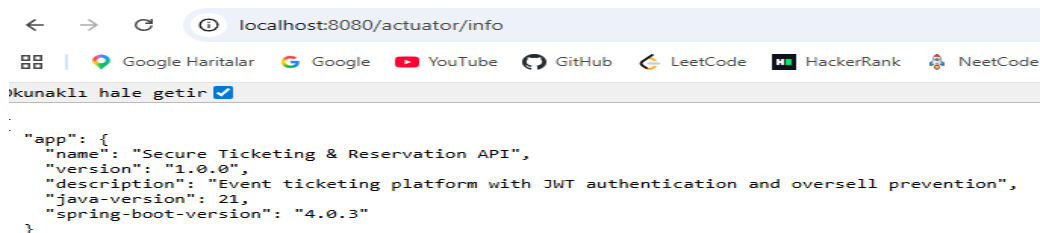
The /actuator/metrics endpoint is intentionally excluded from public exposure to prevent leaking internal system information such as memory usage, thread counts, and connection pool statistics.

<http://localhost:8080/actuator/health>



```
{ "groups": [ "liveness", "readiness" ], "status": "UP" }
```

<http://localhost:8080/actuator/info>



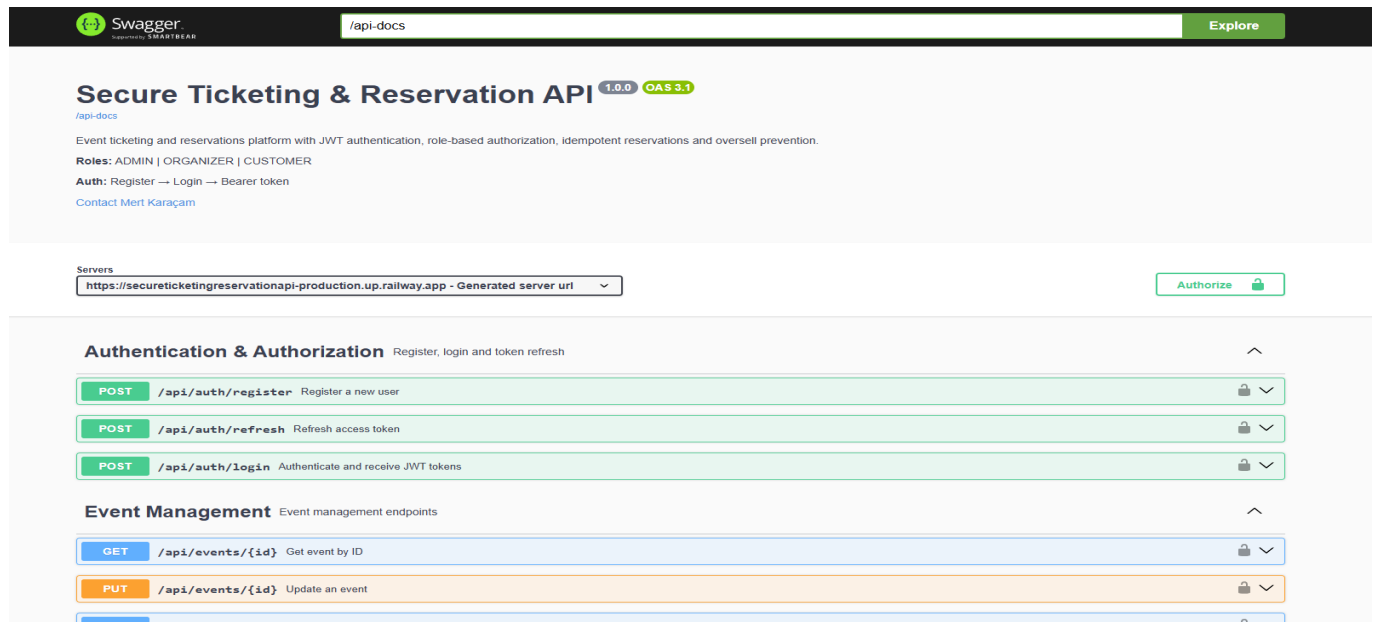
```
{ "app": { "name": "Secure Ticketing & Reservation API", "version": "1.0.0", "description": "Event ticketing platform with JWT authentication and oversell prevention", "java-version": "21", "spring-boot-version": "4.0.3" } }
```

3.4 Public API Documentation

The following links provide direct access to the deployed application and its API documentation.

Live Swagger UI (Railway Deployment)

<https://secureticketingreservationapi-production.up.railway.app/swagger-ui/index.html>



Happy Path — End-to-End Scenario

The following sequence demonstrates the complete reservation lifecycle against the live Railway deployment.

Step 1 — Register as Organizer

```
POST /api/auth/register
```

```
{ "email": "organizer@test.com", "password": "Test@12345!", "role": "ORGANIZER" }  
→ 201 Created: { "id": "c6b72439-...", "email": "organizer@test.com", "roles": "ORGANIZER" }
```

Step 2 — Login and Receive JWT

```
POST /api/auth/login | { "email": "organizer@test.com", "password": "Test@12345!" }  
→ 200 OK: { "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjoxNjU0MjU0MjU0LCJ1aWkiOiJhbm91dCJ9", "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjoxNjU0MjU0MjU0LCJ1aWkiOiJhbm91dCJ9", "tokenType": "Bearer", "expiresIn": 900 }
```

Step 3 — Create and Publish an Event

```
POST /api/events | { "title": "ING Tech Summit 2026", "venue": "Levent Arena Istanbul", "startsAt": "2026-09-10T09:00:00Z", "endsAt": "2026-09-10T18:00:00Z", "capacity": 200 }
```

```
→ 201 Created: { "id": "0bc8bd27-...", "published": false, "capacity": 200 }
```

```
POST /api/events/0bc8bd27-.../publish
```

```
→ 200 OK: { "published": true, "version": 1 }
```

Step 4 — Customer Reserves Seats (with Idempotency-Key)

```
POST /api/events/0bc8bd27-.../reservations | Headers: Idempotency-Key: res-001 |  
{ "seats": 3 }
```

```
→ 201 Created: { "id": "e7eb0874-...", "status": "PENDING", "seats": 3 }
```

Step 5 — Confirm the Reservation

```
POST /api/reservations/e7eb0874-.../confirm
```

→ 200 OK: { "status": "CONFIRMED", "seats": 3 }

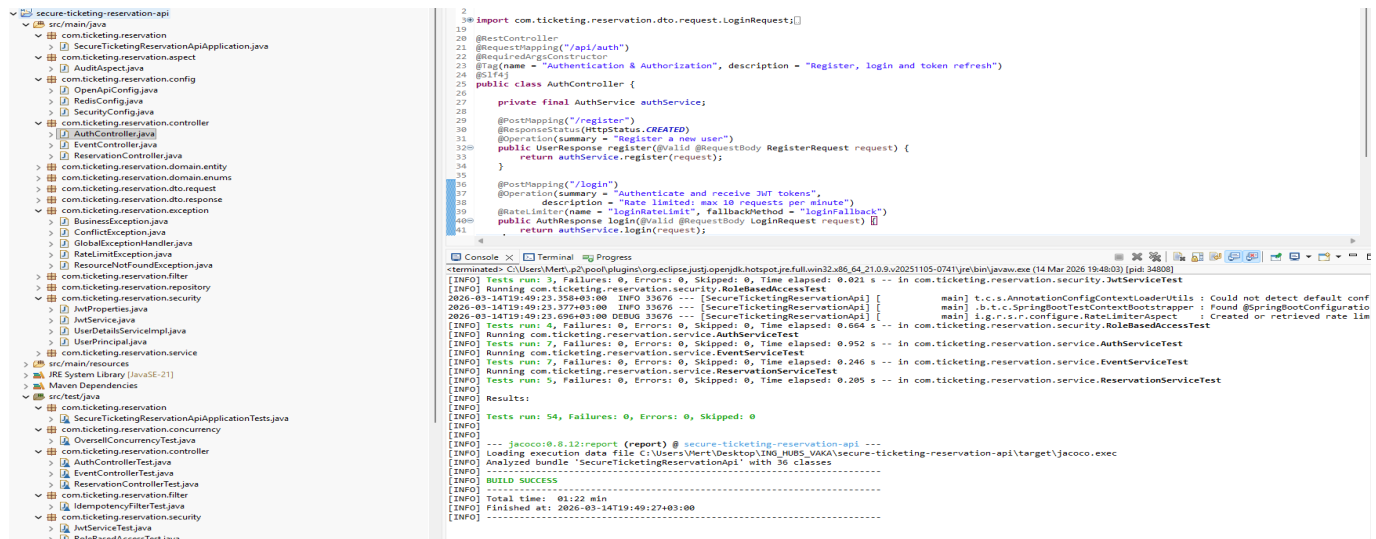
3.5 Testing

Run the full test suite with:

mvn test

To generate the JaCoCo coverage report:

```
mvn test jacoco:report
```



The report is generated at `target/site/jacoco/index.html`.

The test suite covers:

Unit tests

- **ReservationServiceTest** — capacity checks, lock behavior, status transitions
- **EventServiceTest** — ownership validation, published event guard, listing filters
- **AuthServiceTest** — register, login, refresh token flows
- **JwtServiceTest** — token generation, validation, tamper detection
- **IdempotencyFilterTest** — duplicate detection, hash mismatch handling

Integration tests (H2 in-memory)

- **AuthControllerTest** — register, login, wrong password, weak password
- **EventControllerTest** — create, update, publish, list with role enforcement
- **ReservationControllerTest** — create, confirm, cancel with role checks

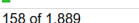
Security tests

- **RoleBasedAccessTest** — CUSTOMER cannot create events, unauthenticated cannot access protected endpoints, public events accessible without token

Concurrency tests

- **OversellConcurrencyTest** — 20 threads competing for 10 seats; asserts total active seats never exceed capacity

3.5.1 Test Coverage

SecureTicketingReservationApi												 Sessions
SecureTicketingReservationApi												
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 com.ticketing.reservation.service		%89		%65	17	51	9	138	4	29	0	4
 com.ticketing.reservation.domain.enums		%50		n/a	1	2	2	4	1	2	1	2
 com.ticketing.reservation.filter		%91		%75	5	16	8	65	0	6	0	2
 com.ticketing.reservation.controller		%87		n/a	2	14	2	14	2	14	0	3
 com.ticketing.reservation.security		%96		n/a	1	18	0	39	1	18	0	3
 com.ticketing.reservation.exception		%94		n/a	1	15	1	21	1	15	0	5
 com.ticketing.reservation		%37		n/a	1	2	2	3	1	2	0	1
 com.ticketing.reservation.aspect		%96		%90	2	15	2	26	0	5	0	1
 com.ticketing.reservation.dto.response		%98		n/a	1	12	1	13	1	12	1	7
 com.ticketing.reservation.config		%100		n/a	0	11	0	47	0	11	0	3
 com.ticketing.reservation.dto.request		%100		n/a	0	5	0	5	0	5	0	5
Total	158 of 1.889	%91	22 of 84	%73	31	161	27	375	11	119	2	36

Created with JaCoCo 0.8.12.202403310830

Current coverage sits above 91% on instruction coverage across all logic classes. The concurrency and integration tests are the most valuable they exercise the full Spring context including the filter chain, security rules, and database interactions.

3.6 Static & Security Code Analysis

Two analysis tools were run against the codebase to catch issues that automated tests alone would not surface code quality problems that can introduce security weaknesses, and bytecode-level security vulnerabilities.

3.6.1 Static Code Analysis (PMD)

PMD performs static analysis without executing the code. From a security perspective, the goal is to eliminate patterns that make the codebase harder to audit and easier to introduce vulnerabilities into dead code, overly broad imports, and inconsistent structure all increase the surface area for mistakes.

```
mvn pmd:check
```

PMD Scan Before:

Last Published: 2026-03-12 | Version: 0.0.1-SNAPSHOT



PMD Results

The following document contains the results of PMD 7.3.0.

Violations By Priority

Priority 4

com/ticketing/reservation/aspect/AuditAspect.java

Rule	Violation	Line
UnnecessaryImport	Unused import 'com.ticketing.reservation.dto.response.Responses.*'	3

com/ticketing/reservation/controller/AuthController.java

Rule	Violation	Line
UnnecessaryImport	Unused import 'com.ticketing.reservation.dto.response.Responses.*'	6

com/ticketing/reservation/controller/EventController.java

Rule	Violation	Line
UnnecessaryImport	Unused import 'com.ticketing.reservation.dto.response.Responses.*'	4

com/ticketing/reservation/controller/ReservationController.java

Rule	Violation	Line
UnnecessaryImport	Unused import 'com.ticketing.reservation.dto.response.Responses.*'	4

com/ticketing/reservation/service/AuthService.java

Rule	Violation	Line
UnnecessaryImport	Unused import 'com.ticketing.reservation.dto.response.Responses.*'	5

PMD Scan After:

Last Published: 2026-03-12 | Version: 0.0.1-SNAPSHOT



PMD Results

The following document contains the results of PMD 7.3.0.

PMD found no problems in your source code.

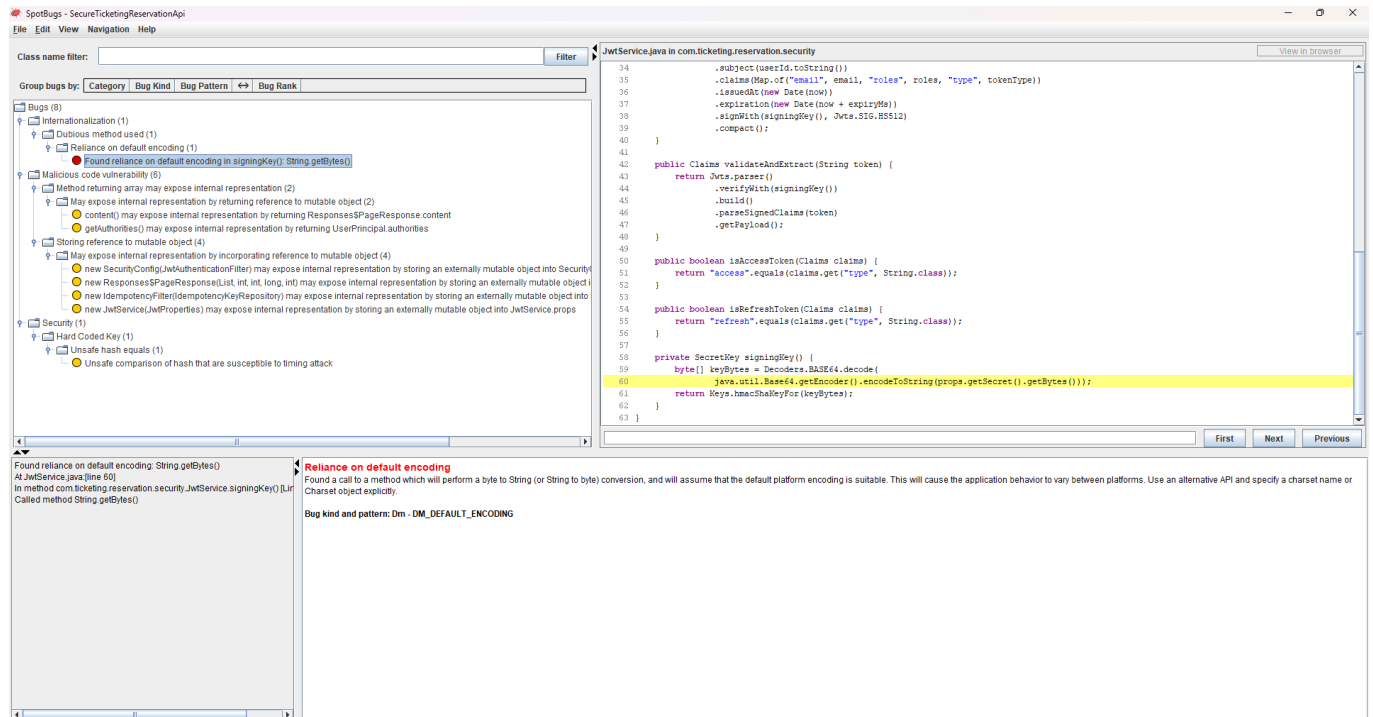
Copyright © 2026..

3.6.2 Secure Code Analysis (SpotBugs + Find Security Bugs)

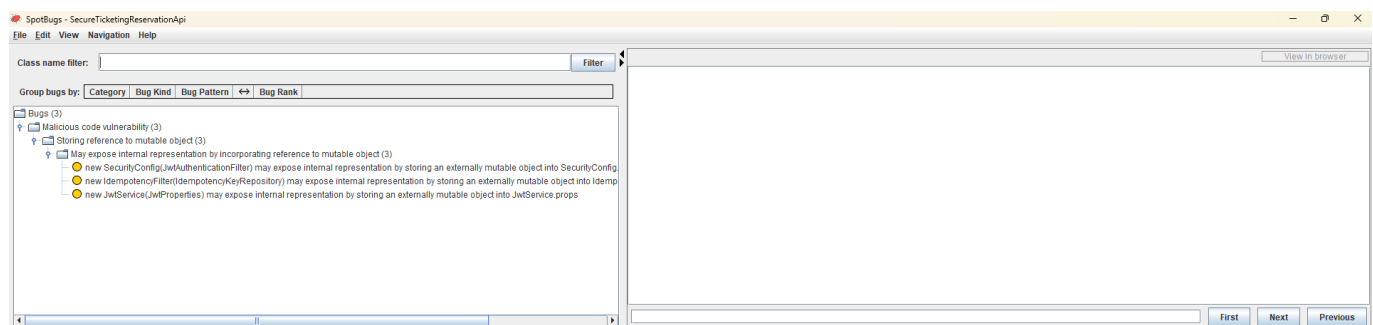
SpotBugs analyzes compiled bytecode and detects patterns that lead to runtime failures and security weaknesses. The Find Security Bugs plugin adds rules mapped to CWE identifiers, specifically targeting issues like timing attacks, unsafe encoding, and mutable object exposure.

```
mvn spotbugs:spotbugs
mvn spotbugs:gui
```

SpotBugs Scan Before:



SpotBugs Scan After:



Analysis results:

- **PMD** -> 0 violations after removing unused imports across 7 files
- **SpotBugs+FindSecure** -> 8 issues identified; 5 resolved, 3 remaining are documented false positives

Resolved issues:

- **DM_DEFAULT_ENCODING** -> JwtService.getBytes() was called without an explicit charset. Fixed by using StandardCharsets.UTF_8.
- **EI_EXPOSE_REP** -> UserPrincipal.getAuthorities() was returning a mutable collection directly. Fixed by wrapping with Collections.unmodifiableCollection().
- **UNSAFE_HASH_EQUALS (CWE-203)** -> IdempotencyFilter was using String.equals() for hash comparison, which is vulnerable to timing attacks. Fixed by using MessageDigest.isEqual().
- **DM_DEFAULT_ENCODING (x2)** -> Additional charset issues resolved in the same pass.

Documented false positives (3 remaining):

Three EI_EXPOSE_REP2 warnings remain on Spring-managed singleton beans (SecurityConfig, JwtService, IdempotencyFilter). SpotBugs flags these because constructor injection stores an external object reference, but since Spring controls the lifecycle of these beans no external mutation is possible. Suppressing them with @SuppressWarnings would hide the signal for future classes, so they are left open with this explanation.

4. Future Improvements

- **Distributed rate limiting:** replace the current Resilience4j in-process limiter with a Redis-backed solution so limits are enforced consistently across multiple instances.
- **Waitlist support:** when an event sells out, allow customers to join a waitlist and be notified automatically when a cancellation frees up seats.
- **Event cancellation:** add organizer-initiated event cancellation with automatic reservation cancellation and notification hooks.
- **Kafka integration:** replace the synchronous reservation flow with an event-driven model so reservation confirmation and audit events are published to a message broker.
- **Token blacklist:** store invalidated tokens in Redis on logout so access tokens cannot be replayed within their remaining TTL.
- **Prometheus / Grafana:** expose custom metrics (reservation rate, cache hit ratio, alarm counts) and hook them into a monitoring dashboard.